

StatFootix

Guide de Présentation Orale

Ce document contient :

Le texte détaillé à dire pour chaque diapositive

Les définitions des termes techniques

Les réponses aux questions pièges

Durée cible : ~20 minutes | ~2 min par diapo

DIAPO 1

Presentation du projet

CE QUE TU DIS

"StatFootix, c'est une application web de statistiques football en temps reel. L'idee de base c'est simple : je voulais pouvoir suivre les matchs en direct, consulter les classements, les resultats et les buteurs, le tout dans une interface propre et rapide."

"Cote technique, j'ai utilise React 18 pour construire l'interface, TanStack Query pour gerer toutes les donnees et le cache, et Netlify Functions comme couche serveur pour ne jamais exposer mes cles API au navigateur."

"Le projet consomme quatre sources de donnees : football-data.org pour les matchs et classements, api-football.com pour les minutes precises en direct, ESPN pour les scores quasi instantanes, et des flux RSS agrege9s pour les actualites."

TERMES CLES A MAITRISER

React 18

Librairie JavaScript pour construire des interfaces web en decoupant l'UI en composants reutilisables. La version 18 apporte le rendu concurrent (transitions plus fluides) et les hooks comme useState, useEffect.

TanStack Query (ex React Query)

Librairie de gestion d'etat serveur : elle s'occupe automatiquement du fetch, du cache, du loading, des erreurs et du refetch. On lui donne une cle (queryKey) et une fonction fetch (queryFn), elle fait le reste.

Netlify Functions

Fonctions serverless qui tournent cote serveur sur les serveurs de Netlify. Elles permettent d'appeler les APIs externes sans exposer les cles dans le navigateur. Pas de serveur a gerer, pas de deploiement complexe.

API (Application Programming Interface)

Interface qui permet a deux logiciels de communiquer. football-data.org expose une API REST : on lui envoie une requete HTTP, elle repond avec des donnees JSON.

SI ON TE POSE DES QUESTIONS

Q : Pourquoi React et pas Vue ou Angular ?

React est le framework le plus utilise en 2024/2025, avec le plus grand ecosysteme. TanStack Query, react-router, tous les outils que j'utilise ont un support React natif. Vue et Angular sont valables mais React etait le choix le plus naturel pour ce projet.

Q : C'est quoi "temps reel" dans ton contexte ?

Ce n'est pas du vrai temps reel (WebSocket). C'est du polling : on interroge les APIs toutes les 30 secondes pendant un match. ESPN repond en moins de 10 secondes, donc on a des scores mis a jour toutes les ~10s en pratique.

DIAPO 2

Architecture : comment les donnees circulent

CE QUE TU DIS

"L'architecture est en trois couches. Premiere couche : le navigateur, c'est ce que l'utilisateur voit.

Deuxieme couche : Netlify, qui joue le role de proxy entre le navigateur et les APIs. Troisieme couche : les APIs externes."

"Le navigateur ne parle JAMAIS directement aux APIs. Il appelle /api/v4/matches, Netlify intercepte, ajoute la vraie cle API depuis une variable d'environnement, et retransmet la reponse. La cle ne quitte jamais le serveur."

"On a quatre sources distinctes : football-data.org pour les donnees structurees (matches, classements, resultats), api-football.com uniquement pour les minutes precises en direct (100 requetes/jour donc on les economise), ESPN pour les scores en quasi-temps-reel, et quatre flux RSS pour les actualites."

TERMES CLES A MAITRISER

Architecture en couches (N-tier)

Separation du code en niveaux independants. Ici : UI (React) / Backend (Netlify Functions) / APIs. Chaque couche ne connait que la couche directement en dessous.

Proxy

Intermediaire qui relaye les requetes. Netlify Function = proxy : recoit la requete du navigateur, la retransmet a l'API externe avec la cle, renvoie la reponse.

Variable d'environnement

Variable stockee cote serveur, invisible du navigateur. Sur Netlify : Settings > Environment variables. Dans le code : `process.env.FOOTBALL_API_KEY`

REST / JSON

REST = style d'API ou chaque URL correspond a une ressource. JSON = format de donnees texte (JavaScript Object Notation), lisible par les humains et les machines.

RSS (Really Simple Syndication)

Format XML standardise pour publier des flux d'articles. Les sites de foot publient leurs articles en RSS, on les consomme et on filtre par mots-cles.

SI ON TE POSE DES QUESTIONS

Q : Pourquoi pas un vrai backend avec Node.js ou Express ?

Pour ce projet, Netlify Functions couvrent exactement le besoin : proxy securise, zero infrastructure a gerer, deploiement automatique a chaque push Git. Un backend custom (Node/Express) apporterait une complexite inutile : serveur a maintenir, certificat SSL, scalabilite... Netlify gere tout ca nativement.

Q : Ca n'est pas une faille de securite de tout passer par Netlify ?

Non, c'est exactement le contraire. Sans Netlify, la cle est dans le code React, donc visible dans le navigateur. Avec Netlify, elle n'est que sur le serveur. C'est le pattern standard pour les apps frontend.

DIAPO 3

Securiser la cle API avec Netlify Functions

CE QUE TU DIS

"Le probleme de base : si tu mets ta cle API directement dans le code React, elle apparait en clair dans le JavaScript qui est envoye au navigateur. N'importe qui peut ouvrir DevTools, chercher la cle, et consommer ton quota a ta place."

"La solution : une Netlify Function qui tourne cote serveur. Le code React appelle /api/v4/matches, Netlify intercepte dans netlify/functions/football.js, ajoute la vraie cle depuis process.env, et retransmet la reponse. La cle ne quitte jamais le serveur."

"Detail important : si l'API repond 429 (trop de requetes), on retransmet ce code tel quel au navigateur. TanStack Query le detecte et conserve les donnees precedentes en cache - pas de spinner, pas de page blanche, l'app reste utilisable."

TERMES CLES A MAITRISER

HTTP 429 (Too Many Requests)

Code d'erreur HTTP signifiant qu'on a depasse le quota de l'API. football-data.org : 10 req/min sur le tier gratuit. api-football.com : 100 req/jour. La reponse inclut souvent un header Retry-After.

DevTools (Outils de developpement)

Panel integre dans tous les navigateurs (F12 ou Cmd+Option+I). Onglet Network : toutes les requetes HTTP. Onglet Sources : le code JavaScript. Une cle dans le code JS y est visible en clair.

netlify.toml

Fichier de configuration Netlify a la racine du projet. Definit les redirects : `[[redirects]] from="/api/*" to="/.netlify/functions/football/:splat"`. Ca permet d'appeler /api/v4/matches depuis React au lieu de l'URL Netlify complete.

Serverless (sans serveur)

Modele ou le code s'execute en reponse a des evenements, sans serveur permanent. Netlify lance la fonction a chaque requete, l'arrete apres. On ne paye que ce qu'on utilise, zero maintenance.

SI ON TE POSE DES QUESTIONS

Q : Et si Netlify tombe, l'app ne marche plus ?

Oui, c'est une dependance. Mais Netlify a un uptime de 99.99%. En pratique, si football-data.org ou l'API tombe, l'app aurait deja des problemes. Le localCache stocke les dernieres donnees valides pour absorber les coupures courtes.

Q : Pourquoi ne pas utiliser CORS directement ?

CORS (Cross-Origin Resource Sharing) est cote serveur des APIs. Meme si l'API autorise CORS, la cle serait toujours dans le code client. Le proxy n'est pas la pour contourner CORS, c'est pour cacher la cle.

DIAPO 4

TanStack Query : le pattern commun

CE QUE TU DIS

"Toute la gestion des données suit le même pattern. Un hook React, une queryKey, une queryFn. La queryKey c'est l'identifiant unique de la donnée en cache."

"Si deux composants appellent le même hook avec la même queryKey, il n'y a qu'UNE seule requête réseau. Par exemple, la page Résultats et le panel Accueil utilisent la même clé : une seule requête pour les deux, cache partagé, synchro instantanée."

"Le staleTime : tant que la donnée est fraîche (pas encore périmée), TanStack ne refetch pas même si le composant est remonte. Ça preserve le quota API. staleTime 20s pour les matchs live, 10min pour les buteurs qui ne changent qu'après un but."

TERMES CLES A MAITRISER

Hook React

Fonction JavaScript commençant par "use" qui encapsule de la logique réutilisable. useQuery, useState, useEffect sont des hooks. Ils ne peuvent être appelés qu'à l'intérieur d'un composant React.

queryKey

Tableau identifiant une donnée en cache. Exemple : ['standings', 'compld'] - si compld change, nouvelle entrée cache, nouveau fetch. Deux hooks avec la même queryKey = même cache = zéro requête en double.

staleTime

Durée en millisecondes pendant laquelle une donnée est considérée "fraîche". Pendant ce temps, TanStack ne refetch pas. Après, la donnée est "périmée" et sera refetchée au prochain montage du composant.

refetchInterval

Polling automatique : TanStack refetch toutes les N millisecondes. Utilise pour les données live. On peut le rendre dynamique : 30s si des matchs sont en cours, 60s sinon.

Cache partagé (shared cache)

TanStack Query maintient un cache global. Même queryKey = même entrée cache pour toute l'app. Pas besoin de Redux ni de Context pour partager les données.

SI ON TE POSE DES QUESTIONS

Q : Pourquoi TanStack Query et pas SWR ou React Context ?

SWR (Stale-While-Revalidate) est bien mais moins complet : pas de mutations, pas de prefetching avancé, pas de devtools intégrés. React Context c'est pour l'état UI (thème, langue), pas pour les données serveur. TanStack est le standard de l'industrie pour les données asynchrones en React.

Q : Et Redux ?

Redux est une solution d'état global, pas spécialisée pour les données serveur. Il faudrait coder manuellement le loading, l'erreur, le refetch, le cache... TanStack le fait nativement en quelques lignes.

DIAPO 5

Recuperer les matchs du jour

CE QUE TU DIS

"Premier probleme decouvert apres le premier deploiement : l'endpoint global de football-data.org sur le tier gratuit ne retourne PAS les matchs de la Coupe du Monde. Donc pour avoir tous les matchs d'une journee, il faut faire deux requetes separees."

"Le delay(700) entre les deux requetes est important : l'API limite a 10 requetes par minute. Sans ce delai, deux requetes quasi-simultanees au chargement peuvent declencher un 429 immediatement."

"Ensuite on deduplique par ID au cas ou un match apparait dans les deux reponses, et on trie chronologiquement par utcDate."

TERMES CLES A MAITRISER

Promise.all

Methode JavaScript qui execute plusieurs promesses en parallele et attend que toutes soient resolues. Plus rapide que d'attendre l'une apres l'autre. Si une echoue, Promise.all rejette (d'ou le try/catch autour).

Rate limiting (limitation de debit)

Restriction du nombre de requetes qu'on peut faire par unite de temps. football-data.org : 10/min sur le tier gratuit. Le 429 (Too Many Requests) est l'erreur renvoyee quand on depasse.

utcDate

Date et heure du match en UTC (Coordinated Universal Time). On la convertit en heure locale pour l'affichage. Utile pour trier les matchs chronologiquement.

Deduplication

Supprimer les doublons. Ici : un match peut apparaitre dans la reponse globale ET dans la reponse WC. On utilise un Set d'IDs pour ne garder que la premiere occurrence.

SI ON TE POSE DES QUESTIONS

Q : Pourquoi ne pas payer le tier superieur pour eviter ces contournements ?

Le but etait de faire tourner l'app gratuitement. Le tier gratuit suffit pour un usage personnel. Le contournement (deux requetes) est propre et fonctionne bien dans la pratique.

Q : Le delay(700) c'est fiable ?

C'est une protection simple mais efficace. On pourrait implementer un retry exponentiel (1s, 2s, 4s...) mais pour ce cas d'usage, 700ms suffit largement pour eviter le rate limiting au chargement initial.

DIAPO 6

Afficher un match : MatchCard

CE QUE TU DIS

"MatchCard c'est le composant de base. Il affiche un match dans n'importe quel contexte : panel du jour, widget live, resultats. Meme composant, donnees differentes."

"Pour le score, on priorise ESPN plutot que football-data.org. ESPN met a jour en moins de 10 secondes, football-data.org a un delai d'environ 1 minute. L'operateur ?. (optional chaining) fait le fallback automatiquement : si ESPN n'a pas le score, on prend celui de football-data.org."

"Pour la minute, calcMinute() calcule cote client depuis des timestamps stockes en localStorage. Elle est mise a jour toutes les 30 secondes via un setInterval sans aucun appel reseau. Selon l'etat du match : 1', 45+2', MT, 73', 90+3', TERMINE..."

TERMES CLES A MAITRISER

Composant React

Fonction JavaScript qui retourne du JSX (HTML-like). Prend des props en entree, affiche de l'UI en sortie. Reutilisable : MatchCard est utilise a plusieurs endroits avec des donnees differentes.

Optional chaining (?.)

Operateur JavaScript ES2020. espnData?.score est equivalent a : si espnData existe alors espnData.score, sinon undefined. Evite les TypeError "Cannot read property of undefined".

setInterval

Fonction JavaScript qui execute du code de facon repete a intervalles fixes. Ici : met a jour la minute affichee toutes les 30 secondes sans refetch reseau. Necessite un cleanup (clearInterval) pour eviter les fuites memoire.

localStorage

Stockage cle-valeur dans le navigateur, persistant entre les sessions. Limite : ~5MB, synchrone (bloquant). On y stocke les timestamps des matchs (kickoffAt, pausedAt, half2Start).

calcMinute()

Fonction utilitaire qui determine la minute a afficher. Priorite : timestamps api-football.com (precis) > estimation pausedAt+15min > heuristique utcDate. Independante du statut API : calcul purement client.

SI ON TE POSE DES QUESTIONS

Q : Pourquoi stocker les timestamps en localStorage et pas en state React ?

Le state React est perdu au rechargement de la page. Si tu recharges pendant un match, tous les timestamps sont perdus et la minute repart de zero. localStorage persiste entre les sessions. Inconvenient : c'est synchrone et limite en taille, mais pour des timestamps c'est largement suffisant.

Q : Comment tu sais que c'est la mi-temps ?

football-data.org renvoie le statut PAUSED. Quand on detecte PAUSED pour la premiere fois, on enregistre pausedAt = Date.now(). calcMinute() retourne "MT" tant que moins de 15 minutes se sont ecoulees depuis pausedAt. Apres 15min, elle bascule automatiquement en 2eme mi-temps cote client, sans dependre du statut API qui peut changer trop tot ou trop tard.

DIAPO 7

Resultats : fetch et affichage

CE QUE TU DIS

"Pour la page Resultats, le point technique interessant c'est le cache partage. La page Resultats et le panel Resultats recents de l'Accueil utilisent exactement le meme hook avec la meme queryKey. Une seule requete reseau pour les deux. Quand la page Resultats se met a jour, le panel Accueil est a jour instantanement, sans code supplementaire."

"Pour l'affichage : les blasons viennent des URLs fournies directement par l'API. Si l'URL est vide, on affiche la premiere lettre du nom de l'equipe. Le gagnant est mis en evidence visuellement avec une classe CSS."

TERMES CLES A MAITRISER

Cache partage TanStack Query

TanStack maintient un QueryCache global accessible a tous les composants. Meme queryKey = meme entree cache. Pas besoin de prop drilling ni de Redux pour partager les donnees entre pages.

Journee / matchday

football-data.org groupe les matchs par journee (matchday). Journee 1, 2, 3... On affiche les matchs groupe par journee, tries du plus recent au plus ancien.

Fallback

Solution de secours quand la donnee principale n'est pas disponible. Ici : si l'URL du blason est vide -> afficher la 1ere lettre du nom. Si ESPN n'a pas le score -> utiliser celui de football-data.org.

SI ON TE POSE DES QUESTIONS

Q : Combien de jours de resultats tu affiches ?

On affiche les 4 derniers jours par default (J-4). C'est parametrable. Football-data.org limite les donnees historiques sur le tier gratuit, donc on reste dans une fenetre courte.

Q : Tu geres le changement de competition ?

Oui. La queryKey contient l'ID de la competition : ['results', compId]. Quand l'utilisateur change de competition, la queryKey change, TanStack fait un nouveau fetch et met a jour automatiquement.

DIAPO 8

Classements : useStandings

CE QUE TU DIS

"Le classement a une complexite particuliere : l'API retourne une structure differente selon la competition. Pour un championnat classique comme la Premier League : un seul tableau. Pour la Coupe du Monde ou la Ligue des Champions : plusieurs groupes, Groupe A, B, C..."

"Le hook detecte ca automatiquement. Si la reponse contient plusieurs groupes (`realGroups.length > 1`), on est en mode multi-groupes, sinon tableau unique. Le composant Classement.jsx n'a pas besoin de savoir quelle competition est chargee, il recoit groups et standings et s'adapte."

TERMES CLES A MAITRISER

Standings (classement)

Tableau de classement avec pour chaque equipe : position, matchs joues, victoires, nuls, defaites, buts pour/contre, difference de buts, points.

Multi-groupes

Competitions avec phase de groupes : CdM, LDC, Europa League. L'API renvoie un tableau de groupes, chaque groupe contenant son propre classement. Structure differente de celle d'un championnat standard.

`realGroups.length > 1`

Detection automatique du type de competition. Si l'API renvoie plus d'un groupe, on est en mode poule. Pas besoin de coder en dur quel competition est "multi-groupes".

SI ON TE POSE DES QUESTIONS

Q : Comment tu affiches le bracket (arbre) de la Coupe du Monde ?

Le bracket est genere avec du SVG (Scalable Vector Graphics) via un calcul mathematique. Les matchs de phase finale ont des IDs previsibles, on les positionne dans un arbre avec des coordonnees calculees. Les lignes de connexion sont des paths SVG generees automatiquement.

DIAPO 9

Top buteurs : useScorers

CE QUE TU DIS

"Un detail technique interessant ici : le parametre enabled de TanStack Query. Si aucune competition n'est selectionnee (compld est null), enabled vaut false et la requete ne part pas du tout. Pas besoin d'un if dans le composant, TanStack gere ca nativement."

"Le staleTime est a 10 minutes parce que les stats de buteurs ne changent qu'apres un but. Pas besoin de rafraichir toutes les 30 secondes comme les scores live."

TERMES CLES A MAITRISER

enabled (TanStack Query)

Booleen qui active ou desactive une query. enabled: !!compld signifie : activer seulement si compld est truthy (non-null, non-vide). Pendant que la query est desactivee : pas de fetch, isLoading reste false.

Scorers / Buteurs

Endpoint football-data.org : /v4/competitions/{id}/scorers. Retourne les 20 premiers buteurs de la competition avec nom, equipe, nombre de buts, penaltys.

staleTime strategique

Adapter le staleTime a la frequence de changement de la donnee. Score live : 20s. Resultats : 2min. Classement : 5min. Buteurs : 10min. Plus le staleTime est long, moins on consomme de quota.

SI ON TE POSE DES QUESTIONS

Q : Tu affiches combien de buteurs ?

Top 20 via l'API, mais l'UI met en avant visuellement les 3 premiers (podium : or, argent, bronze). Les autres sont listes en-dessous.

DIAPO 10

Scores en direct : overlay ESPN

CE QUE TU DIS

"Le vrai probleme du temps reel sur le tier gratuit de football-data.org : environ 1 minute de delai sur les mises a jour de score. Pour un match en direct, c'est trop long."

"Solution : ESPN a une API publique non officielle qui met a jour en moins de 10 secondes. On la poll toutes les 10 secondes uniquement quand un match est en cours. ESPN devient un overlay : il ne remplace pas football-data.org (qui reste la source de verite pour les competitions, classements...), il enrichit juste les scores en direct."

"Le challenge c'est le fuzzy matching : ESPN appelle les equipes differemment de football-data.org. 'PSG' vs 'Paris Saint-Germain'. La solution : normaliser les noms (minuscules, accents retires) et comparer les 5 premiers caracteres. 'paris' correspond a 'paris saint-germain'. Ca marche pour 99% des cas."

TERMES CLES A MAITRISER

Polling

Technique consistant a interroger une API a intervalles reguliers pour detecter des changements. Oppose au push (WebSocket, Server-Sent Events) ou le serveur notifie le client. Polling ESPN : toutes les 10s quand un match est actif.

Overlay de donnees

Superposition d'une source de donnees sur une autre. football-data.org = source de verite (structure, meta-donnees). ESPN = couche de fraicheur pour les scores. Le composant fusionne les deux.

Fuzzy matching

Correspondance approximative de chaines de caracteres. Ici : normalisation + comparaison des N premiers caracteres. Pas d'algo sophistique (Levenshtein), juste du pragmatisme : les 5 premiers caracteres du nom normalis\xe9 suffisent dans la pratique.

ESPN API (non officielle)

ESPN expose une API publique non documentee via `site.api.espn.com`. Elle n'a pas d'authentification, mais n'a pas de garantie de stabilite. Si ESPN change son API, l'overlay tombe mais le reste de l'app fonctionne.

Normalisation de chaine

`.toLowerCase()` + suppression des accents via `normalize('NFD')` + `replace`. Exemple : "Paris Saint-Germain" -> "paris saint-germain" -> "paris". "PSG" -> "psg". Les 5 premiers chars ne matchent pas, mais en realite ESPN utilise "paris".

SI ON TE POSE DES QUESTIONS

Q : L'ESPN API ne va pas casser un jour ?

C'est un risque accepte. C'est une API publique non officielle sans garantie. Si elle casse : l'app continue de fonctionner, juste avec les scores de football-data.org (1min de delai). C'est exactement pour ca que c'est un overlay et pas la source principale.

Q : Le fuzzy matching ne peut pas se tromper et matcher la mauvaise equipe ?

En theorie oui. En pratique les 5 premiers caracteres sont assez discriminants pour les equipes de grandes competitions. Un faux positif theorique : deux equipes dont les noms commencent pareil. Ca n'est pas arrive sur les competitions couvertes.

DIAPO 11

Conclusion

CE QUE TU DIS

"Pour resumer les quatre points cles que j'ai retenus de ce projet :"

- Le proxy Netlify est non-negociable des qu'il y a une cle API : une ligne dans netlify.toml et la cle ne quitte plus le serveur.
- TanStack Query simplifie enormement la gestion des donnees : un seul pattern couvre le fetch, le cache, le loading, l'erreur et le refetch automatique.
- ESPN comme overlay est une solution pragmatique quand l'API principale est trop lente : on garde football-data.org comme source de verite et ESPN pour la fraicheur.
- enabled et staleTime font qu'on ne brule pas le quota : zero requete inutile, chaque appel API est justifie.

"Ce projet m'a appris que la gestion des donnees en temps reel dans une app frontend, c'est surtout une question de compromis : fraicheur vs quota, precision vs fiabilite, source unique vs sources multiples."

TERMES CLES A MAITRISER

Source de verite (single source of truth)

Principe d'architecture : une seule source fait autorite pour une donnee. Football-data.org est la source de verite pour les matchs. ESPN est un enrichissement, pas une source de remplacement.

Quota API

Limite du nombre de requetes autorisees par periode. football-data.org : 10/min, pas de limite journaliere sur le free tier. api-football.com : 100/jour sur le free tier. Depasser = 429. Gerer le quota = optimiser staleTime + refetchInterval.

SI ON TE POSE DES QUESTIONS

Q : L'app est deployee en production ?

Oui, sur Netlify. Le deploiement est automatique a chaque push sur la branche main. Netlify detecte que c'est un projet React (Vite), build automatiquement (npm run build) et deploie le dossier dist/.

Q : Tu as des tests ?

Pas de tests automatises dans ce projet. C'est une limite identifiee. En production, j'ajouterais des tests unitaires sur les fonctions critiques (calcMinute, fuzzy matching, deduplication) avec Vitest ou Jest.

Q : Quelles sont les limites de l'app ?

1/ Le tier gratuit des APIs limite l'usage a un cadre personnel (pas scalable). 2/ ESPN API non officielle, sans garantie. 3/ Pas de PWA : pas de support offline (mis a part le localCache). 4/ Pas de tests automatises.

Q : Tu utiliserais quoi differemment si tu le refaisais ?

J'ajouterais TypeScript des le debut pour typer les reponses API. J'ecrirais des tests unitaires sur les utils. Et je separerais encore plus les couches : un fichier de types, un fichier de constantes, des schemas Zod pour valider les reponses API.

QUESTIONS PIEGES — AIDE-MEMOIRE

Q : C'est legal d'utiliser l'API ESPN ?

ESPN ne documente pas son API publique mais ne l'interdit pas non plus. L'app est a usage personnel, pas commercial. Si ESPN bloque un jour, l'app continue avec football-data.org.

Q : Pourquoi pas GraphQL ?

football-data.org expose uniquement du REST. GraphQL aurait du sens si j'avais un backend custom qui agregait les sources. Pour du pure consumption d'APIs REST externes, REST + TanStack Query est plus simple.

Q : Tu gerez l'authentification ?

Non, l'app est publique sans compte. Les cle API sont uniquement cote serveur dans les variables d'environnement Netlify. Si je voulais des comptes utilisateurs, j'utiliserais Netlify Identity ou Clerk.

Q : Et la performance ? Lighthouse score ?

React avec Vite en prod = bundle minifie et tree-shaken. TanStack Query evite les refetch inutiles. Les images sont des URLs externes donc dependent de l'API. Le vrai goulot d'etranglement c'est la latence des APIs externes, pas le code.

Q : localStorage c'est pas dangereux pour les timestamps ?

Pas pour des timestamps de matchs publics. localStorage est accessible a tout JS sur le domaine, mais il n'y a rien de sensible ici (pas de donnees utilisateur). Le vrai risque serait d'y stocker des tokens d'auth (il ne faut pas).